

Fachinformatiker/in

Lernfeld 7 – Cyber-physische Systeme ergänzen

Durchführung eines Projektes mit Hilfe des Arduinos

Projekthandout

Arduino PIXEL – Handheld-Spielekonsole

Teilnehmer: Malte Linnemann, Marcel Schmidt, Jakob Münch

Datum: 29. Mai 2026

Klasse: IFA41

Betreuer: Herr Diekbreder

0.1 Thema

Im Rahmen dieses Projekts wurde eine tragbare Spielekonsole mit einem Arduino Mega 2560 entwickelt. Die Konsole trägt den Namen *Arduino PIXEL*. Sie verfügt über ein OLED Display, einen Joystick und zwei Tasten zur Steuerung. Auf der Konsole können mehrere selbst programmierte Spiele gespielt werden.

0.2 Projektidee und Motivation

Ziel des Projekts war es, die im Unterricht behandelten Inhalte praktisch anzuwenden. Dazu gehören unter anderem die Nutzung von Displays, das Auslesen von Sensoren, die Verarbeitung von Tastereingaben und die Programmierung von Mikrocontrollern.

Die Idee einer Spielekonsole wurde gewählt, weil dabei viele verschiedene Funktionen kombiniert werden können. Gleichzeitig bietet das Projekt genügend Herausforderungen, um die eigenen Programmierkenntnisse zu erweitern. Dazu zählen zum Beispiel die Verwaltung mehrerer Spielzustände, die Verarbeitung von Benutzereingaben und die Entwicklung einer übersichtlichen Softwarestruktur.

0.3 Projektziele

Die fertige Konsole sollte folgende Anforderungen erfüllen:

- Startbildschirm mit Auswahl der verfügbaren Spiele
- Mehrere spielbare Spiele auf einem Gerät
- Pausenmenü mit den Optionen Weiter, Neustart und Zurück zum Hauptmenü
- Speicherung von Highscores auch nach dem Ausschalten
- Akustische Rückmeldungen über einen Buzzer
- Einfache Erweiterung um weitere Spiele
- Flüssiges Spielerlebnis ohne sichtbare Verzögerungen

1 Ressourcen und Ablaufplanung

1.1 Sachmittel und Kostenplanung

Komponente	Anzahl	Kosten in € (ca.)
Arduino Mega 2560 (Rev. 3)	1	50
2.42" OLED-Display (SSD1309, SPI)	1	18
Joystick-Modul (VRX, VRY, SW)	1	3
Taster (Pull-Down)	2	0,50
Aktiver Buzzer	1	1
Widerstände, Jumper-Kabel, Steckbrett	–	5
LEGO-Steine (Gehäuse)	–	5
Gesamtkosten:		82,50

Tabelle 1: Hardware-Komponenten und Kosten

1.2 Zeitliche Ablaufplanung

Der Bearbeitungszeitraum erstreckte sich vom 22.04.2026 bis zum 03.06.2026. Die folgende Tabelle gibt einen Überblick über die wesentlichen Projektphasen.

Phase	Tätigkeit	Stunden
1	Konzeption: Anforderungen definieren, Hardware auswählen, Spieleideen skizzieren	5
2	Basisaufbau: Display in Betrieb nehmen, Joystick und Taster anbinden, Software-Grundgerüst	3
3	Spieleentwicklung: Homescreen, Space Invaders, Dino Jump	3
4	Erweiterung: Vampire Survivors, Snake, Eternal	3
5	Gehäusebau, Feinabstimmung, Pause-Logic, Highscore-EEPROM	3
6	Dokumentation, Präsentation, Videoerstellung	3

2 Durchführung

2.1 Vorgehensweise und Prozessschritte

Zu Beginn wurden die benötigten Hardwarekomponenten aufgebaut und einzeln getestet. Dabei wurde geprüft, ob Display, Joystick, Taster und Buzzer korrekt funktionieren.

Anschließend wurde das Grundgerüst der Software entwickelt. Dabei wurde darauf geachtet, dass die einzelnen Spiele voneinander getrennt bleiben. Dadurch können neue Spiele später einfacher hinzugefügt werden.

Die Umsetzung erfolgte in mehreren Schritten:

1. Aufbau und Verkabelung aller Hardwarekomponenten.
2. Einrichtung des Displays sowie Einlesen der Eingaben über Joystick und Taster.
3. Entwicklung eines Hauptmenüs zur Auswahl der Spiele.
4. Programmierung der einzelnen Spiele.
5. Einbau einer Pausenfunktion für alle Spiele.
6. Speicherung der Highscores im internen Speicher des Arduino.
7. Testen und Optimieren der gesamten Konsole.
8. Bau eines passenden Gehäuses aus LEGO.

Während der Entwicklung wurde das Projekt regelmäßig getestet. Fehler konnten dadurch früh erkannt und behoben werden.

2.2 Schaltplan / Visualisierung

Ein detaillierter Schaltplan mit allen Pin-Verbindungen befindet sich in der Anlage. Die zentrale Pin-Belegung ist:

Komponente	Pin	Typ	Hinweis
OLED CS	10	Digital	SPI Chip-Select
OLED DC	9	Digital	SPI Data/Command
OLED RST	8	Digital	Display-Reset
OLED MOSI	51	HW-SPI	Hardware SPI
OLED SCK	52	HW-SPI	Hardware SPI
Joystick X	A0	Analog	X-Achse (links/rechts)
Joystick Y	A1	Analog	Y-Achse (hoch/runter)
Joystick SW	2	Digital	Taster im Joystick
Button 1	32	Digital	Aktion (Schießen/Springen/Bestätigen)
Button 2	30	Digital	Pause / Zurück
Buzzer	40	Digital	Aktiv (HIGH = ein)

2.3 Aufgetretene Schwierigkeiten

- **Displaygröße:** Ursprüngliches Display zu klein, daher Wechsel auf ein größeres OLED Display.
- **Zu wenig Speicher:** Arduino Uno bot nicht genügend Speicher, daher Wechsel auf den Arduino Mega 2560.
- **Pausenfunktion:** Umsetzung des zuverlässigen Pausierens und Fortsetzens war komplexer als erwartet.
- **Gehäusebau:** Integration aller Komponenten im LEGO Gehäuse erforderte mehrere Anpassungen.

2.4 Abweichungen und Anpassungen

Im Verlauf des Projekts wurden einige Änderungen vorgenommen. Statt des ursprünglich geplanten Arduino Uno kam ein Arduino Mega 2560 zum Einsatz. Zusätzlich wurde ein größeres Display verwendet.

Diese Änderungen verbesserten die Leistung des Systems und ermöglichten die Umsetzung weiterer Spiele. Dadurch wurde der Funktionsumfang des Projekts sogar erweitert.

2.5 Ausblick

Die entwickelte Konsole kann zukünftig um weitere Funktionen ergänzt werden.

- Entwicklung zusätzlicher Spiele
- Betrieb mit einem Akku für mehr Mobilität
- Neues Gehäuse aus dem 3D Drucker
- Verbesserte Soundausgabe mit Lautsprecher

3 Projektergebnis

3.1 Soll-Ist-Vergleich

- **Homescreen:** Erfüllt – Spielauswahl mit Vorschau, Icons und Animationen.
- **Spieleanzahl:** Übertroffen – 5 Spiele statt der geplanten 2.
- **Highscore:** Erfüllt – dauerhafte Speicherung und Anzeige der Bestwerte.
- **Pause und Menü:** Erfüllt – Pausieren, Fortsetzen und Neustarten möglich.

- **Sound:** Erfüllt – akustische Rückmeldungen über einen Buzzer.
- **Performance:** Erfüllt – flüssige Darstellung ohne spürbare Verzögerungen.
- **Erweiterbarkeit:** Erfüllt – neue Spiele können einfach ergänzt werden.
- **Gehäuse:** Erfüllt – kompakte und tragbare Bauweise.

3.2 Qualitätskontrolle

Die Konsole wurde in mehreren Test-Sessions auf Herz und Nieren geprüft: sämtliche Spiele wurden auf korrektes Verhalten bei Game-Over, Pause und Menü-Rückkehr getestet. Die Highscore-Persistenz wurde durch Stromtrennung verifiziert. Zusätzlich wurde kontrolliert, ob das Gehäuse stabil ist und alle Bedienelemente zuverlässig funktionieren.

3.3 Abweichungen und Anpassungen

Die wesentlichen Abweichungen vom ursprünglichen Plan – Umstieg auf ein größeres Display und den leistungsfähigeren Mega 2560 – haben den Projektumfang eher erweitert als eingeschränkt. Der größere Speicher ermöglichte die Entwicklung aufwendigerer Spiele wie Vampire Survivors und Eternal. Diese Spiele benötigen mehr Speicher, da sie viele Objekte und Spielzustände gleichzeitig verwalten.

3.4 Ausblick

Das Projekt bietet zahlreiche Anknüpfungspunkte für Weiterentwicklungen:

- **Weitere Spiele:** Die modulare Architektur erlaubt das einfache Hinzufügen von Tetris, Pong oder Jump-'n'-Run-Varianten.
- **Akku-Betrieb:** Ein Li-Ion-Akku mit Ladeelektronik würde das Gerät vollständig mobil machen.
- **Gehäuse-Veredelung:** Ein 3D-gedrucktes, ergonomisch optimiertes Gehäuse böte mehr Komfort und eine professionellere Optik.
- **Ton-Verbesserung:** Ein passiver Lautsprecher mit PWM-Ansteuerung könnte Melodien statt einfacher Pieptöne erzeugen.

4 Anlage

Folgende Materialien sind diesem Handout beigelegt bzw. liegen im Moodle-Abgabeordner vor:

1. Schaltplan
2. Interessante Code Stellen
3. Screenshots von Spielen und Menüs

5 Anlagen

5.1 Anlage 1 – Schaltplan

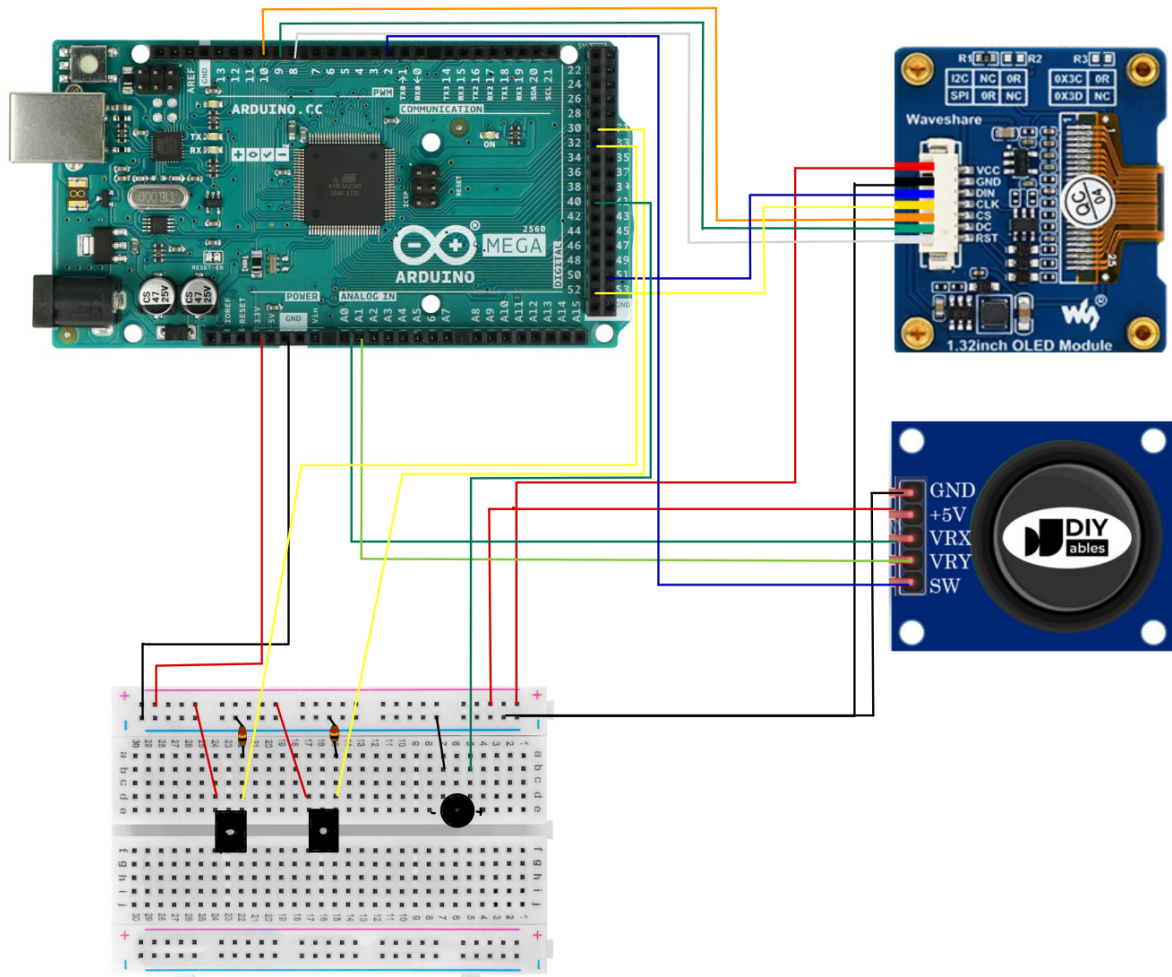


Abbildung 1: Schaltplan der Arduino-PIXEL-Konsole

5.2 Anlage 2 – Arduino PIXEL und Spiele

Homescreen



Abbildung 2: Der Startbildschirm

Spielauswahl



Abbildung 3: Implementierung der Spielauswahl

5.3 Anlage 3 – Interessante Code-Stellen

Setup

```
void setup() {
    initHardware();
    soundManager.init(PIN_BUZZER);

    // Init input buttons
    input.btn1.init(PIN_BTN1, INPUT);
    input.btn2.init(PIN_BTN2, INPUT);
    input.joyButton.init(PIN_JOY_SW, INPUT_PULLUP);

    // Register button pointers for ISR access
    _btn1Ptr = &input.btn1;
    _btn2Ptr = &input.btn2;
    _joyBtnPtr = &input.joyButton;
    enableButtonInterrupts(PIN_JOY_SW, PIN_BTN1, PIN_BTN2);

    // Register games
    homescreen.addGame(&originGame, ICON_ORIGIN);
    homescreen.addGame(&spaceInvaders, ICON_SPACE_INVADERS);
    homescreen.addGame(&dinoJump, ICON_DINO_JUMP);
    homescreen.addGame(&vampireSurvivors, ICON_VAMPIRE_SURVIVORS);
    homescreen.addGame(&eternalGame, ICON_ETERNAL);

    homescreen.setup();
    spaceInvaders.setup();
    dinoJump.setup();
    vampireSurvivors.setup();
    originGame.setup();
    eternalGame.setup();

    // Seed random from unconnected analog pin + runtime
    randomSeed(analogRead(A2) ^ analogRead(A3) ^ micros());

    soundManager.beep(30);
}
```

Abbildung 4: Setup Funktion

Programm Loop

```
void loop() {  
    readInputs(input);  
  
    unsigned long now = millis();  
    if (now - lastFrameMs >= FRAME_INTERVAL_MS) {  
        lastFrameMs = now;  
  
        if (appState == AppState::HOMESCREEN) {  
            GameResult result = homescreen.loop(input);  
            if (result == GameResult::GAME_SELECTED) {  
                currentGame = homescreen.getSelectedGame();  
                if (currentGame) {  
                    currentGame->setup();  
                    appState = AppState::PLAYING_GAME;  
                    soundManager.beep(50);  
                }  
            }  
        } else {  
            GameResult result = currentGame->loop(input);  
            if (result == GameResult::EXIT_TO_MENU) {  
                currentGame = nullptr;  
                appState = AppState::HOMESCREEN;  
                homescreen.setup();  
                soundManager.beep(30);  
            }  
        }  
    }  
  
    soundManager.update();  
}
```

Abbildung 5: Übergeordneter Programm Loop